Technical cheat sheet

Brilliante Barbershop - AI-assisted booking, kiosk queue and hairstyle preview system. Answers below are what the code actually does.

1. Receipts

Do we issue receipts?

Yes, a **digital payment reference**. Every captured payment stores a `transaction_ref` in `appointment_payments`; for online payments this is the PayMongo reference. The customer sees it as *Receipt: <ref>* on the appointment and on the completed-booking detail screen, and the receptionist sees the same reference in the portal.

If pressed: It is a reference number, not a printed or emailed receipt, and not a BIR Official Receipt. The shop still issues its own OR at the counter - that is a business-registration requirement, outside the system's scope.

Can a customer get proof after the fact?

Yes. The reference persists on the booking record, so it can be read back any time from the app and cross-checked against PayMongo and the `appointment_payments` row.

2. Encryption and data protection

How is data encrypted?

Three layers, each doing a different job:

Layer	How
In transit	HTTPS/TLS on the API (Render) and the portal (Vercel); the database connection forces `sslmode=require`, so app-to-Postgres traffic is encrypted too.
At rest	Supabase/AWS encrypt the database volume and the storage bucket at the platform level.
Passwords	Never stored by us. Supabase Auth hashes them (bcrypt). Staff accounts use PHP `password_hash()`.
Sessions	Supabase-issued JWT, verified server-side on every customer request.
Webhooks	PayMongo callbacks are verified with **HMAC-SHA256** against a shared secret, so a forged 'payment succeeded' is rejected.

Are names and phone numbers individually encrypted?

No, and we do not claim they are. They are ordinary columns protected by full-disk encryption, TLS, and role-based access. The accurate phrasing is "stored in an access-controlled database with encryption in transit and at rest" - not "column-level encryption".

If pressed: Field-level encryption would block the search, sort and join the booking system depends on. The mitigation is access control plus audit logging, which is the standard trade-off for operational data.

What about the face photos? (Data Privacy Act, RA 10173)

Consent is captured before the first scan. Scan captures and generated previews auto-purge after **90 days** (`SCAN_MEDIA_RETENTION_DAYS`); inactive accounts after **3 years**, and only when purging is explicitly switched on.

If pressed: Be precise: images are *retained for the retention window*, not deleted immediately after display - the barber needs to see what the customer picked. Do not describe it as process-and-discard.

Who can see what?

Three roles - customer, barber, receptionist/admin - each gated server-side. Staff actions are written to an audit log (`audit_log()`) covering user, service, hairstyle and settings changes, plus refunds.

3. Tools and versions

Component	Version / detail
Mobile app	Flutter 3.44.0 (stable), Dart 3.12.0
Android	compileSdk 36, targetSdk 36, **minSdk 24** (Android 7.0 and up)
iOS	Codebase is Flutter, so iOS-capable; not yet built or signed
Backend API	PHP 8.2 on `php:8.2-apache` (Docker), ~70 endpoints
Database	PostgreSQL 17.6 (Supabase), 34 tables
Auth + storage	Supabase Auth (JWT) and Supabase Storage
AI service	Python + FastAPI, OpenCV, **MediaPipe 0.10.x**
Face detection (device)	Google ML Kit via `google_mlkit_face_detection`
Image generation	Grok Imagine (xAI) - fallbacks: Gemini 2.5 Flash Image, Replicate FLUX Kontext Pro
Text AI	Gemini 2.5 Flash Lite (chatbot, admin insights)
Payments	PayMongo (GCash / card), webhook-confirmed
Hosting	API on Render, receptionist/admin portal on Vercel
Portal	Plain HTML/CSS/JavaScript, no framework or build step
Version control	Git / GitHub, with a scheduled DB backup workflow

4. Architecture, in one breath

How do the pieces fit?

Three clients - customer app, barber app, kiosk (all one Flutter codebase) - plus a **browser portal** for the receptionist and admin. All four talk to **one PHP API**, which owns every business rule and is the only thing that touches **PostgreSQL**. A separate **Python service** handles face and hair analysis. Image generation goes out to a third-party model.

If pressed: Why PHP and not a framework: the rules live in one place and are readable without a build step, which matters for a system a shop has to maintain.

Why a separate Python service?

MediaPipe is Python-only. Keeping it in its own service means a failure there degrades the scan gracefully instead of taking the booking system down.

5. What the AI actually does

Where is the AI?

Scanning: ML Kit on the phone frames the face and takes 3 captures. **Analysis:** MediaPipe server-side - Face Landmarker (478 points) for face shape, Image Segmenter for a per-pixel hair mask giving texture, colour and coverage, majority-voted across the 3 captures. **Matching:** deterministic rules, not a model. **Preview image:** generative AI edits your own photo.

So the recommendation is not AI?

Correct, and this is a design choice. The *perception* is AI; the *matching* is a weighted rule scorer (face-shape match +0.45, texture match +0.55, penalties for styles the hair cannot take). It is explainable - the system can state why a style was recommended - and it can only ever suggest a haircut the shop actually offers, with a price and a duration.

If pressed: A learned ranker over a 32-style catalog would be unexplainable and could recommend something unbookable. Honest framing: **AI perceives, rules match, AI renders.**

Is the preview a real photo of me?
It is an **edit of your own capture**, not a generated stranger. The prompt forbids changing face, skin, eyes, glasses, clothing and background, and specifies the haircut in barber terms - fade height, guard numbers, length on top, fringe, neckline.

What happens if someone is bald or wearing a cap?
Detected and handled. Hair coverage under 1% reads as bald, 1-3% as obstructed. The system refuses to invent a hair texture, says which case it is, and still gives face-shape-based recommendations.
If pressed: This was a real bug we found and fixed: a shirt over the head once returned "Hair: Wavy". It now reports "not measured".

6. Hair colour detection

How does it read my hair colour?
It samples the pixels inside the hair mask - only hair, not skin or background - takes the **median** colour, converts it to HSV and maps it to a plain label: *black, dark brown, brown, blonde, red, gray*. The front photo is preferred because it is the best lit.

Why the median and not the average?
Hair has shiny highlights far brighter than the hair itself, and averaging them in lifted black hair to "dark brown". On real captures the mean read V=59-65 while the median read V=40-46 - exactly the difference between black and dark brown. The median ignores the highlights.

What is the colour used for?
One job only: telling the image model to **keep your colour**. The prompt says preserve the natural colour and do not lighten, darken, redden or recolour. It does not change which haircuts are recommended.
If pressed: Limits: it reads colour under the light you scanned in, so a dim room shifts brown towards black. Dyed or bleached hair is the weakest case.

7. Custom haircuts and the catalog limit

What if a customer wants a haircut not in the library?
They can still book it. Every service has a **free-text instructions box** at booking, and whatever they type reaches the barber on the appointment detail screen. What they do not get is an **AI preview** of it, because a preview needs a written prompt we control.

Why is the catalog limited at all?
Because a catalog entry is not just a picture. It carries a **price, a duration, barber instructions and clipper sizes**. Those feed the slot engine and the bill. A free-form style has none of them, so the system cannot schedule it or charge for it correctly.
If pressed: 32 styles across 3 face shapes and 3 hair textures is 92 catalog variants - it covers the shop's actual menu, not every haircut that exists.

Why not let AI invent the recommendation instead of using a library?
An invented style is **unbookable** - no service ID, no duration for the slot engine, no instructions for the barber holding the clippers. And it could not be explained to the customer. The library is what the shop can actually cut.

Could you add a style?
Yes - an admin adds it through the portal. The honest cost is that a new style needs its own written prompt, or the preview would be wrong. A test enforces that: every catalog style must have one.

8. Group booking, in simple words

How does booking for a group work?
You say how many people. The system looks for **that many free hours in a row with one barber** - 4 people needs 4 straight hours - and offers up to **10 start times** spread over at least 3 days, so it reads as a choice of time, not one take-it-or-leave-it slot.

What if no single barber has that many hours free?
Two fallbacks, both offered as a choice: **Split** - everyone at the *same* hour with a different barber each. **Waves** - when the group is bigger than the number of barbers, as many as there are free barbers go first, the rest in the next hour. 5 people and 4 barbers becomes 4 at 10:00 and 1 at 11:00.

Is it one booking or many?
One payment, many rows. Each person is a separate appointment with their own barber, time and position in the group, tied together by a group ID. That is what lets a split group run under four different barbers and still be one bill.
If pressed: A partial booking - fewer people than asked for - is never automatic. The customer must tick a box confirming it books fewer.

9. Services

What is a service here?
A priced, timed item the shop sells. Each one carries a **price**, a **base duration**, a category, and a **channel** - appointment or walk-in - which decides where it appears. The duration drives the slot engine; the price drives the bill and the refund amount.

Service	Price / time / channel
Exclusive Haircut	P300.00, 60 min, appointment
Regular Haircut	P250.00, 35 min, walk-in

Who manages them?
The admin portal - add, edit, activate or deactivate. Changes are written to the audit log.
If pressed: Deleting is a **deactivation**, not a row delete, and deliberately so: past appointments reference the service, and hard-deleting it would break the history and the revenue figures.

10. Admin settings

What can the shop change without a developer?
The things that change in a real shop:

Setting	Current value
Opening hours	09:00-19:00, per weekday, with Sunday closed
Break window	12:00-13:00 (no slots generated)
Closed dates	Holidays and one-off closures
Slot length	60 minutes
Appointment / walk-in time	60 min / 35 min
Timezone	Asia/Manila
Queue merge	On - one queue for walk-ins and appointments
Services, hairstyles, staff	Full add / edit / deactivate

Why does the timezone setting matter?
The servers run on UTC. Without it, the Philippine morning still counts as yesterday, so 'today' would be wrong until 8am. Every date boundary in the system is computed in the shop's timezone, not the server's.

11. Notifications and reminders

How do notifications work?
In-app only. Events write a row to a notifications table; the app's bell icon reads them and shows an unread count. Six kinds: Appointment, Reminder, Queue, Promotion, System, Payment. Staff get their own feed for cancellations, refunds and queue changes.

Is there a reminder for an upcoming appointment?
Partly, and this is worth being exact about. There is one reminder: if an appointment is coming up and is *still unpaid*, the customer is nudged once before the booking is written off. There is **no** general "your appointment is tomorrow" reminder - which is why that toggle is hidden in the settings screen rather than shown and doing nothing.
If pressed: Two honest reasons. First, no push notifications: that needs Firebase, which was deliberately excluded, so a reminder only appears when the app is opened. A reminder you only see after opening the app is not a reminder. Second, the sweeps run on request traffic rather than a scheduler, so timing is best-effort. A dependable reminder needs push plus a cron - that is the clearest next step for this system.

12. Known limits - say these before you are asked

What is the accuracy of the face-shape detection?
We do not publish an accuracy figure, because we have not run a labelled validation study. The landmarks are measured; the *classification* into face shapes uses published ratio thresholds.
If pressed: The `confidence_score` of 0.88 in the catalog is a **seeded per-style value, not a model confidence** - it is nearly identical on every variant, which is exactly why we removed the percentage from the UI rather than show a number that looks measured and is not.

Weak spots you should own
Bald vs dark cap is the hardest case - both read as near-zero hair. **Bleached or dyed hair** can breach the brightness threshold. **Hair length** is inferred from the lowest hair pixel relative to the chin, so a dark collar can read as long hair - which is why length no longer gates anything user-visible.

What is not built?
Push notifications (would need Firebase - deliberately excluded), multi-branch support, light mode, and an automated background scheduler. Time-based sweeps run on request, so they catch up the next time a staff screen loads.

13. Reliability, money and abuse

Concern	Control
Double booking	Slot locking at the database level; a paid booking holds its slot and conflicting writes are rejected.
Abandoned payments	A 10-minute reservation hold, then the slot is released automatically.
Refund abuse	3 refunds in 24h triggers a 7-day cooldown; 5 in 30 days triggers 30 days. Cancelling is never blocked - only the refund.
Late cancellation	Inside 24 hours the booking still cancels (freeing the chair) but is not refundable. Stated on the payment screen before paying.
Refund payout	Manual by the receptionist - there is no gateway auto-refund - and the desk sees a warning badge for repeat refunders.
Queue identity	Kiosk walk-ins get a claim code, so an overheard queue number is not enough to claim someone else's slot.
Barber leave	24 paid days a year, pending requests included, enforced on both request and approval.
Data loss	Daily automated `pg_dump` of the public schema via GitHub Actions to a private Supabase bucket.

14. How we know it works

How is it tested?
24 PHP test scripts covering booking rules, refunds, leave balances, group bookings, the queue merge, audit logging and the AI prompt catalog - about 250 assertions, run against the live database inside transactions that are rolled back. Plus **16 Flutter test files** (111 tests) and a clean static analysis.

Give an example of a bug a test caught
Seven of the 32 catalog hairstyles had no AI prompt and silently rendered as a generic barbers cut - no error, just the wrong haircut. A test now asserts that every catalog style resolves to its own prompt, and an unmatched style raises a visible error instead of guessing.

15. Scope and contribution

Will it scale?
Sized for one shop: 4 barbers, hourly slots, a 30-day booking window. Postgres and the stateless API would carry far more; the real limit is the single-branch data model, which is a deliberate scope boundary, not an oversight.

Which SDG does this serve?
SDG 8 (Decent Work and Economic Growth) primarily - it formalises a small local business's scheduling, payments and records. **SDG 9 (Industry, Innovation and Infrastructure)** secondarily, as digital adoption by a micro enterprise.

What is genuinely new here?
Not the booking system. It is **linking a measured facial analysis to a catalog the shop can actually cut**, and showing the customer a preview on their own face before they commit - which turns "what do you want?" from a conversation into a shared picture.

Generated from the live codebase. Figures - versions, limits, retention windows and test counts - were read from the toolchain, source and config, not from memory. Regenerate with `python docs/make_defense_cheatsheet.py`.